







Layer 1 — Version 3: `fetch(..., { cache: "no-store" })`

Now we make **one tiny change**:

```
export default async function ProductsPage() {
  const response = await fetch(
    "https://api.example.com/products",
    {
      cache: "no-store"
    }
  )

  const products = await response.json()

  return (
    <div>
      <h1>Products</h1>

      <ul>
        {products.map((item) => (
          <li key={item.id}>
            {item.name}
          </li>
        ))}
      </ul>
    </div>
  )
}
```

This one option changes our mental model significantly.



1 What does `cache: "no-store"` mean?

It means:

Don't store/reuse this fetch result in the Data Cache.

So instead of:

```
fetch()
  ↓
Data Cache
  ↓
products
```



we have:

```
fetch()  
  ↓  
API  
  ↓  
products
```



Every time this fetch executes, it needs to obtain fresh data rather than reuse a Data Cache entry.

2 Does this mean the component executes on every request?

For our **traditional Layer-1 mental model**, yes.

Because the page needs fresh API data:

```
Request #1  
  ↓  
execute Page()  
  ↓  
fetch API  
  ↓  
render  
  ↓  
response
```



Then:

```
Request #2  
  ↓  
execute Page()  
  ↓  
fetch API  
  ↓  
render  
  ↓  
response
```



And again:

```
Request #3  
  ↓  
execute Page()  
  ↓
```



```
fetch API
  ↓
render
  ↓
response
```

So we're now in the classic **dynamic/request-time rendering** mental model.

3 What happened to the Full Route Cache?

This is the important consequence.

Previously:

```
Data Cache
  ↓
render
  ↓
Full Route Cache
```



Now:

```
Request
  ↓
fetch no-store
  ↓
API
  ↓
render
  ↓
response
```



The route is **not stored in the Full Route Cache as a reusable static rendering** in this traditional model.

So conceptually:

```
Full Route Cache
  X
```



4 Let's trace an actual request

User enters:

```
/products
```



Browser sends:

```
GET /products
```



Potentially:

```
Browser
  |
  ▼
CDN
```



Now here's an important distinction.

CDN ≠ Full Route Cache

Even though the route isn't in Next.js's Full Route Cache, the request could still encounter an HTTP/CDN cache depending on the deployment and HTTP caching headers.

But for a truly request-specific/no-store response, you generally don't want an intermediary reusing that response.

So for our mental model, think:

```
Browser
  |
  ▼
CDN / HTTP layer
  |
  ▼
Next.js
```



5 Next.js receives the request

Next.js executes:

```
ProductsPage()
```

Then:

```
const response = await fetch(  
  "https://api.example.com/products",  
  {  
    cache: "no-store"  
  }  
)
```

Because of `no-store` :

Data Cache



So:

```
Next.js  
  |  
  ▼  
fetch()  
  |  
  ▼  
API  
  |  
  ▼  
Fresh response
```



6 Then your `map()` runs

Suppose the API returns:

```
[  
  { "id": 1, "name": "iPhone" },  
  { "id": 2, "name": "MacBook" }  
]
```

Your code executes:

```
products.map(item => (  
  <li>{item.name}</li>  
))
```

which produces the rendered UI:

Products

iPhone

MacBook



Then Next.js produces the response.

Conceptually:

Request



Page executes



fetch()



API



products



map()



render



HTML + RSC information



HTTP Response



Browser



7 Compare this with Version 2

This is the most important comparison so far.

Version 2 — cacheable fetch

fetch()



Data Cache



render



Full Route Cache



HTTP Response



Version 3 — no-store

```
fetch({ cache: "no-store" })  
  ↓  
✗ Data Cache  
  ↓  
render at request time  
  ↓  
✗ reusable Full Route Cache  
  ↓  
HTTP Response
```



🔥 That's the difference.

8 Let's see two users

Suppose User A visits at 10:00:

```
User A  
  ↓  
Next.js  
  ↓  
API → [iPhone, MacBook]  
  ↓  
render  
  ↓  
HTML
```



Then the database changes:

```
API now:  
[iPhone, MacBook, iPad]
```



User B visits at 10:01:

```
User B  
  ↓  
Next.js  
  ↓  
API → [iPhone, MacBook, iPad]  
  ↓  
render  
  ↓  
HTML
```



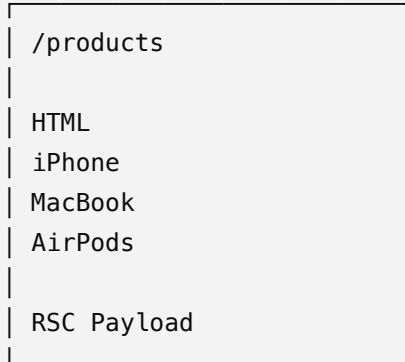
So each request can get the current API result.

That's the purpose of `no-store`.

9 What does the Full Route Cache look like now?

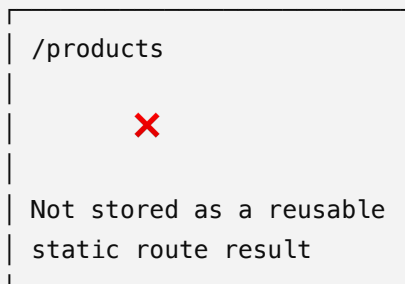
Previously:

Full Route Cache



With `no-store`, in our Layer-1 model:

Full Route Cache



Instead, the result is produced:

REQUEST

↓

RENDER

↓

RESPONSE

10 But wait — what about the Router Cache?

This doesn't mean the browser can't cache anything.

This is where you must keep the caches separate.

Even though:

Data Cache ❌
Full Route Cache ❌



the client-side App Router can still have:

Router Cache
↓
RSC Payload



So:

SERVER SIDE



Data Cache ❌
Full Route Cache ❌

CLIENT SIDE

Router Cache ✅ potentially

That's why we should never say:

"no-store" means nothing is cached anywhere."

It specifically changes the **server-side fetch/data caching** and, in the traditional model, prevents that route from being statically reused through the Full Route Cache.

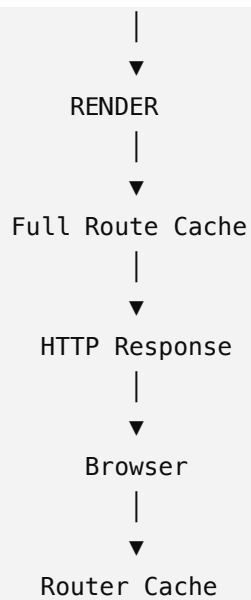


The complete picture

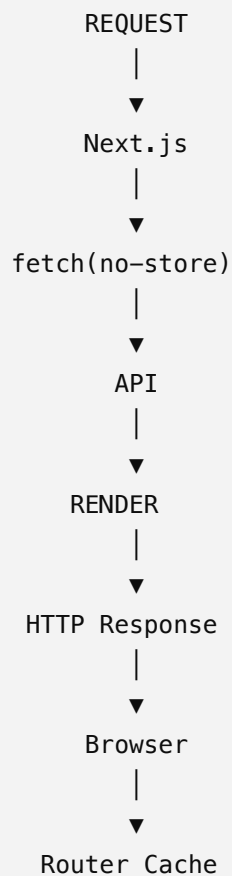
Cacheable fetch

REQUEST
↓
Next.js
↓
fetch
↓
Data Cache





no-store fetch



Notice the big difference:

cacheable fetch:



API → Data Cache → render → Full Route Cache

no-store:

API → render → response

Our six questions

Question	cache: "no-store"
When does data fetch?	Request time
Is fetch result in Data Cache?	✗ No
Is route in Full Route Cache?	✗ Not as a reusable static result in Layer 1
When does rendering happen?	Request time
What does browser receive?	HTTP response with rendered output
Router Cache?	Can still exist independently

And now comes the subtle question

You might reasonably ask:

"If `no-store` means fresh API data on every request, then is `no-store` basically the same thing as SSR?"

That's our next concept.

We'll distinguish:

SSG
SSR
ISR



from:

Data Cache
Full Route Cache
Router Cache



because these are **not four names for the same thing**.

Once that distinction clicks, Next.js rendering becomes *way* less mysterious.



ok

Exactly 👍 Now let's make one of the **most important distinctions in Next.js**.

🚀 Layer 1 — Rendering vs Caching

You've now seen:

```
cacheable fetch
  ↓
Data Cache
  ↓
possibly static rendering
  ↓
Full Route Cache
```



and:

```
fetch({ cache: "no-store" })
  ↓
fresh data
  ↓
request-time rendering
```



Now we need to separate two concepts that are often taught together:

SSG / SSR / ISR are rendering strategies.

Data Cache / Full Route Cache / Router Cache are caching mechanisms.

They interact, but they're not the same thing.

1 SSG — Static Site Generation

SSG essentially means:

Generate the page ahead of the user's request.

Example:

</> TypeScript



```
export default function Page() {  
  return <h1>Hello</h1>  
}
```

Conceptually:

BUILD



↓

Render Page

↓

HTML + RSC Payload

↓

Full Route Cache

Then:

USER REQUEST



↓

reuse generated result

↓

HTTP Response

So:

Rendering time → Build time



That's the core idea behind SSG.

2 SSR — Server-Side Rendering

Now consider:

</> TypeScript



```
export default async function Page() {  
  const response = await fetch(  
    "https://api.example.com/products",  
    {  
      cache: "no-store"  
    }  
  )  
  
  const products = await response.json()
```

```
return <ProductList products={products} />
}
```

The data must be fresh.

So:

```
REQUEST
  ↓
Next.js
  ↓
fetch API
  ↓
render page
  ↓
HTML + RSC
  ↓
HTTP Response
```



Rendering happens **during the request**.

That's the traditional SSR mental model.

```
Rendering time → Request time
```



3 ISR — Incremental Static Regeneration

Now imagine:

```
</> TypeScript
export const revalidate = 60
```



You're basically saying:

"This page can be statically generated, but it shouldn't remain fresh forever."

Conceptually:

```
BUILD
  ↓
Render
  ↓
Full Route Cache
  ↓
Serve cached result
```



```
↓  
60 seconds...  
↓  
stale  
↓  
regenerate  
↓  
new cached result
```

So ISR sits between the two ideas:

```
SSG  
|  
| generated ahead of time  
|  
▼  
ISR  
|  
| regenerated periodically  
|  
▼  
SSR  
request-time rendering
```



But don't think of this as a strict spectrum. They're different rendering/caching behaviors.



Here's the key distinction

Consider:

SSG



What are we talking about?

Rendering timing:

BUILD TIME



Now consider:

SSR



What are we talking about?

Rendering timing:

REQUEST TIME



But:

Data Cache
Full Route Cache
Router Cache



are answering:

"Where can information/results be reused?"

Different question.

4 Let's put them together

SSG

```
graph TD
    BUILD --> RENDER
    RENDER --> FullRouteCache[Full Route Cache]
    FullRouteCache --> REQUEST
    REQUEST --> reuseResult[reuse result]
```



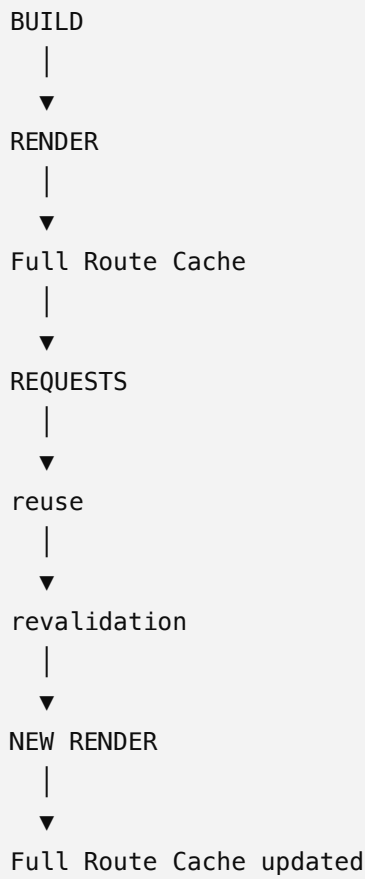
SSR

```
graph TD
    REQUEST --> RENDER
    RENDER --> HTTPResponse[HTTP RESPONSE]
```



No reusable Full Route Cache result in the traditional model.

ISR



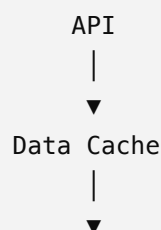
5 Now where does Data Cache fit?

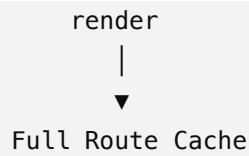
Suppose ISR page has:

```
</> TypeScript

const products = await fetch(
  "https://api.example.com/products",
  {
    next: { revalidate: 60 }
  }
)
```

Now we can have:





So **both the data and the rendered route can have caching/revalidation behavior.**

This is why we mustn't say:

"ISR = Data Cache."

No.

ISR is describing **how the rendered route gets regenerated.**

6 An example makes this clearer

Imagine an e-commerce page:

/products



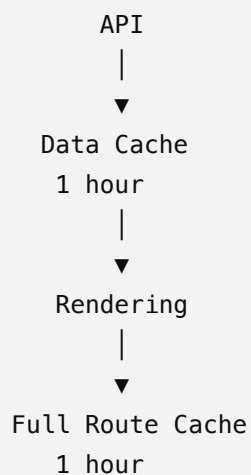
The API changes every hour.

You decide:

revalidate = 3600



Conceptually:



A user visits:

10:00 → cached page

10:15 → cached page



10:45 → cached page
11:00 → revalidation opportunity

That's the ISR-style idea.

7 Now compare with SSR

If we instead have:

```
</> TypeScript

fetch(url, {
  cache: "no-store"
})
```

then:

```
User A
  ↓
API
  ↓
render
  ↓
response

User B
  ↓
API
  ↓
render
  ↓
response

User C
  ↓
API
  ↓
render
  ↓
response
```

That's request-time rendering.

🔥 But here's an interview-level nuance

Don't say:

"no-store" IS SSR."

A better statement is:

In the traditional App Router model, an uncached/request-time data dependency such as `fetch(..., { cache: 'no-store' })` can cause the route to be dynamically rendered, meaning its HTML is generated at request time. This is the classic SSR mental model.

Why the careful wording?

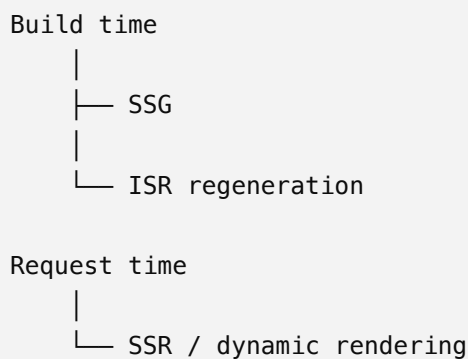
Because Next.js's rendering and caching architecture has evolved, especially with Cache Components.

That's exactly why we'll eventually learn Layer 2.

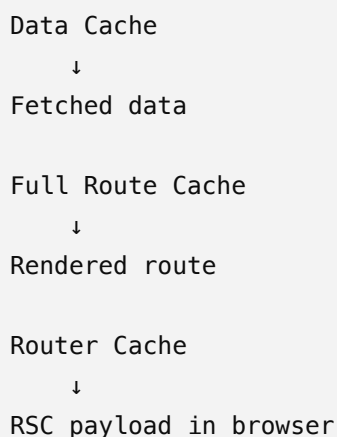
8 Our mental model so far

We can now separate the architecture into two dimensions.

Dimension A — WHEN do we render?



Dimension B — WHAT can be cached?

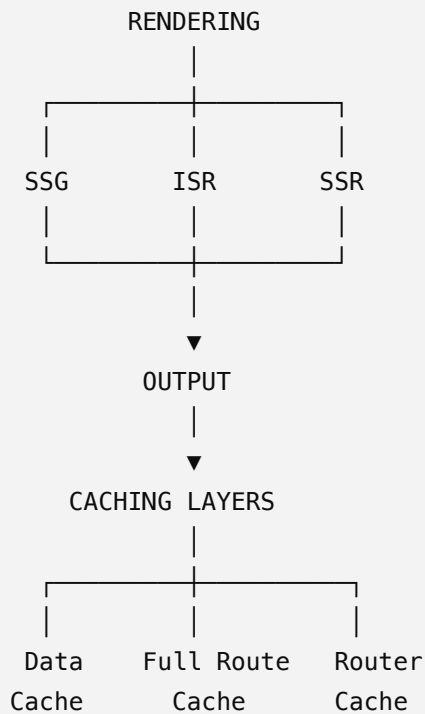


CDN / HTTP Cache



HTTP response at intermediary

Put them together:



The big takeaway

If an interviewer asks:

"What is SSR?"

Think:

Where/when does rendering happen?

At request time.

If they ask:

"What is the Data Cache?"

Think:

Where is fetched data reused?

Server-side data cache.

If they ask:

"What is the Full Route Cache?"

Think:

Where is the rendered route result reused?

Server-side rendered route cache.

If they ask:

"What is Router Cache?"

Think:

What does the client-side App Router keep?

RSC payloads in the browser.



Next: Version 4 — revalidate

Now we're ready for the fun one.

We'll take:

```
</> TypeScript
```

```
fetch(url)
```



and introduce:

```
</> TypeScript
```

```
fetch(url, {  
  next: {  
    revalidate: 60  
  }  
})
```



Then we'll trace **exactly what happens at 10:00, 10:30, 11:00, and 11:01**—including **Data Cache vs Full Route Cache** and when the actual rendering happens.

That will make **ISR finally click**.









Ahhh, yes — I see the exact confusion now. 😊 You're asking:

"If `/products` is cached and doesn't depend on the cookie, and I change a cookie in the browser, how does the server ever learn that the cookie changed?"

The answer is: **it learns it on the next request that actually reaches the server.** And if that request is satisfied entirely by a CDN cache, the origin server may **not need to know at all.**

Let's separate these cases.

1. Cookie changes in the browser

Suppose:

Browser

theme = light



Server Action changes it:

```
cookies().set("theme", "dark")
```



Response:

```
</> http
```



```
Set-Cookie: theme=dark
```

Browser now has:

```
Browser
```



```
theme = dark
```

At this point:

```
Server: ❌ hasn't received "theme=dark" as a new request
```



```
Browser: ✅ knows theme=dark
```

2. Then user requests `/products`

Browser makes:

```
</> http
```



```
GET /products
```

```
Cookie: theme=dark
```

Now there are two possibilities.

Case A — CDN handles it

```
Browser
```



```
| GET /products
```

```
| Cookie: theme=dark
```



```
CDN
```

```
| Cache HIT
```



```
Cached /products response
```

The request **doesn't reach Next.js origin**.

Therefore:

Next.js server

✗ doesn't learn about this request

✗ doesn't need to



The CDN has already answered the request.

And that's perfectly fine if `/products` doesn't care about `theme`.

3. So when does Next.js actually learn the new cookie?

When there is a request that **reaches Next.js**.

For example:

Browser



Cookie: theme=dark



CDN



Cache MISS



Next.js



cookies()



Now Next.js can see:

theme=dark



So:

Cookie changed



Browser stores new cookie



Future HTTP request



Cookie automatically attached



If request reaches server



↓
Server can read new cookie

4. But here's the important realization 💡

You don't actually need to **"tell the server that the cookie changed"** as a separate operation.

The cookie itself is the state.

The browser effectively says:

"From now on, whenever I make an applicable request, I'll send `theme=dark`."

So imagine:

10:00

Browser:
theme=light



Server Action:

Set-Cookie: theme=dark



Browser:

theme=dark



Then:

10:05
GET /profile
Cookie: theme=dark



Server sees it.

There was no separate:

UPDATE_COOKIE /api/notify-server



required.

5. What if `/products` is completely cached?

Suppose:

`/products`



doesn't care about cookies.

CDN has:

`/products → Product HTML`



User requests it:

Browser
Cookie: theme=dark
|
▼
CDN
|
▼
Cached Product HTML



Next.js doesn't learn about the cookie.

And that's okay.

Because the page doesn't use it.

It's exactly like sending someone a letter that says:

"Here is my favorite color: dark"



but the person only needs to answer:

"What products are available?"



They don't need to process your favorite color. 😊

6. Now compare with `/profile`

Suppose `/profile` says:

`</> TypeScript`



```
const cookieStore = await cookies()
const theme = cookieStore.get("theme")?.value
```

Now the cookie matters.

Browser:

```
</> http

GET /profile
Cookie: theme=dark
```

If CDN cannot safely serve a generic cached response:

```
Browser
  ↓
CDN
  ↓
Next.js
  ↓
cookies()
  ↓
theme = dark
  ↓
render
```

Now Next.js **does** learn about it.

7. There's one more scenario: you WANT the server to know immediately

Suppose you have some client-side cookie that you change using JavaScript:

```
</> JavaScript

document.cookie = "theme=dark"
```

The server isn't immediately notified.

If you specifically need the server to know immediately, you make a request:

```
</> JavaScript

document.cookie = "theme=dark"

await fetch("/api/theme", {
```

```
method: "POST"  
})
```

Now:

```
Browser  
|  
| Cookie already changed  
| Cookie: theme=dark  
▼  
POST /api/theme  
▼  
Next.js  
|  
▼  
Server knows theme=dark
```

But normally you don't need this extra request unless your application has a reason to synchronize server-side state.

This is the BIG distinction

There are actually **three events**:

Event 1 — Cookie changes

```
Server → Set-Cookie → Browser
```

or JavaScript changes it.

Event 2 — Browser makes a request

```
Browser → HTTP Request + Cookie → network
```

Event 3 — Somebody processes that request

Could be:

```
Browser  
↓  
CDN  
↓  
response
```

or

Browser
↓
CDN
↓
Next.js



If CDN answers it:

Next.js doesn't see it.



If CDN forwards it:

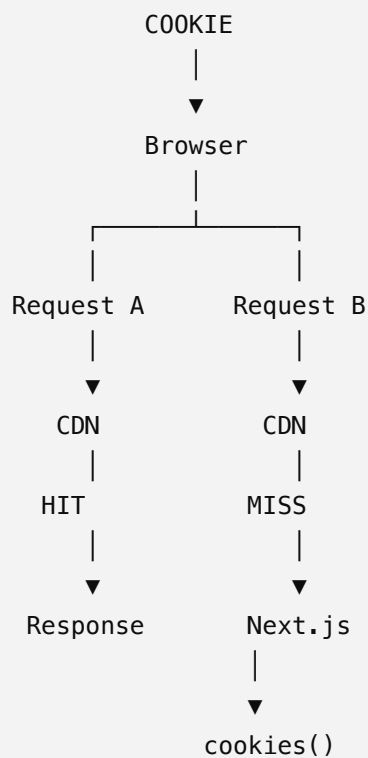
Next.js sees the cookie.



The sentence that should make this click

A cookie is not a live connection between the browser and server. It's client-side state that gets included in applicable HTTP requests.

So:



In **Request A**, Next.js never knows the cookie existed.

In **Request B**, Next.js sees the cookie.

And that's completely normal. 👍

The next thing that will make this whole topic click is understanding **what exactly a CDN cache key is and whether cookies are part of that key**—because that determines whether two users with different cookies can receive the same cached response.



ok

Perfect 🤔 Let's lock this piece in.

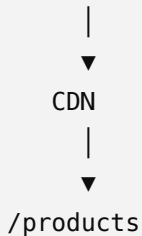
🧠 Next: CDN Cache Key + Cookies

The question we'll answer is:

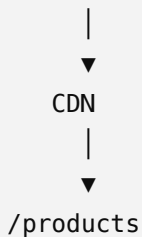
If two users request `/products` but have different cookies, how does the CDN know whether it can give them the same cached response?

We'll trace:

User A
Cookie: theme=dark



User B
Cookie: theme=light



Then we'll understand:

1. What a cache key actually is
2. Why `/products` normally maps to one cached response
3. How `Vary` works
4. Whether cookies can be part of cache variation
5. Why personalized pages generally shouldn't be blindly CDN-cached

6. How this differs from **Next.js Data Cache / Full Route Cache / Router Cache**

7. Finally, we'll return to our **Layer 1 static vs dynamic exercise**. 🚀



ok

Alright 😊 let's do it. This is one of those concepts that makes **CDN caching + cookies** suddenly click.

🌐 **CDN Cache Key — What actually identifies a cached response?**

Imagine the CDN receives:

```
</> http
GET /products
```



The CDN needs to answer:

"Do I already have a response for this request?"

It uses something called a **cache key**.

A simplified cache key might be:

```
https://example.com/products
```



So:

```
Request
GET /products
  ↓
Cache Key
example.com/products
  ↓
CDN Cache
  ↓
Cached Response
```



1. Two users request the same URL

Suppose:

User A

```
GET /products  
Cookie: theme=dark
```



User B

```
GET /products  
Cookie: theme=light
```

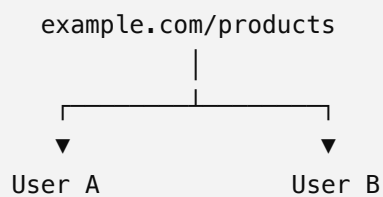


If the CDN's cache key is simply:

```
example.com/products
```



then both requests map to:



So the CDN could potentially return the **same cached response** to both.

That's perfectly fine **if the response doesn't depend on the cookie**.

2. Example where this is perfectly safe

Your `/products` page:

</> TypeScript



```
export default async function ProductsPage() {  
  const products = await getProducts()  
  
  return (  
    <ul>  
      {products.map(product => (  
        <li key={product.id}>{product.name}</li>  
      ))}  
    </ul>  
  )  
}
```

```
)  
}
```

The page doesn't care about:

```
theme  
session  
language  
user ID
```



So:

```
User A  
Cookie: theme=dark  
  |  
  ▼  
/products  
  |  
  ▼  
CDN  
  |  
  ▼  
Cached products response
```



User B:

```
User B  
Cookie: theme=light  
  |  
  ▼  
/products  
  |  
  ▼  
CDN  
  |  
  ▼  
SAME cached products response
```



That's fine.

Because:

```
Cookie —X→ Products HTML
```



The cookie doesn't influence the response.

3. Now let's make the cookie matter

Suppose:

`</>` TypeScript

```
export default async function ProductsPage() {  
  const cookieStore = await cookies()  
  
  const theme = cookieStore.get("theme")?.value  
  
  return (  
    <div className={theme}>  
      Products  
    </div>  
  )  
}
```

Now:

Cookie
↓
theme
↓
HTML

User A:

theme=dark
↓
<div class="dark">

User B:

theme=light
↓
<div class="light">

So one cached response cannot represent both.

4. Here's the problem with a naive CDN

Suppose CDN already has:

Cache:

```
/products  
↓  
<div class="dark">
```

User B arrives:

```
</> http  
  
GET /products  
Cookie: theme=light
```



If CDN says:

"Oh, /products is cached! I'll return it."

then:

```
User B  
theme=light  
  
↓  
  
CDN  
  
↓  
  
<div class="dark">
```



🔥 Wrong response.

That's a **cache correctness problem**.

5. So how does a CDN handle this?

There are several mechanisms in real systems.

One important HTTP mechanism is:

```
</> http  
  
Vary
```



For example:

```
</> http
```



Vary: Accept-Language

This tells caches:

"The response varies based on this request header."

So conceptually:

```
/products + language=en  
/products + language=fr
```



can be treated as different variants.

6. What about cookies?

Here's where things get more nuanced.

You *can* design caching behavior around cookies, but you generally don't want to casually make:

```
Cache Key =  
URL + entire Cookie header
```



because imagine:

```
Cookie:  
session=abc123;  
theme=dark;  
cart=92837;  
...
```



Now every user's session could produce a completely different cache entry.

You'd destroy much of the benefit of shared caching.

You could theoretically get:

```
/products + userA cookie  
/products + userB cookie  
/products + userC cookie  
/products + userD cookie  
...
```



That's basically **personalized caching**.

7. This is why personalized pages are tricky

Imagine:

```
</> TypeScript

const session = cookies().get("session")

return <h1>Welcome, {sessionUser.name}</h1>
```

User A:

Welcome Gopi

User B:

Welcome Rahul

You absolutely cannot have:

```
/products
↓
Welcome Gopi
```

as one generic shared CDN response.

Otherwise:

```
User B
↓
CDN
↓
"Welcome Gopi"
```

😞 That's obviously terrible.

So systems generally avoid treating such personalized responses as ordinary shared public cache entries, or explicitly vary/cache them according to an appropriate strategy.

8. Now connect this to Next.js

This is why our earlier statement was:

```
cookies()
↓
```

```
request-dependent
↓
dynamic rendering
```

Under the **traditional Layer 1 model**, Next.js doesn't say:

"I'll just put this personalized page in the generic Full Route Cache."

Instead, the route is treated as request-dependent.

Conceptually:

```
Request
↓
Next.js
↓
cookies()
↓
render THIS request
↓
response
```



9. But here's a very important distinction 🚨

Don't think:

"If a request contains a cookie, CDN can never cache it."

That's too simplistic.

The correct idea is:

The presence of a cookie doesn't automatically make every response uncacheable. What matters is whether the response is safe to reuse for another request.

For example:

```
Cookie: theme=dark
```



but `/products` doesn't use theme.

The response could still potentially be shared.

Whereas:

Cookie: userId=123



and the response says:

Welcome Gopi



That's personalized.

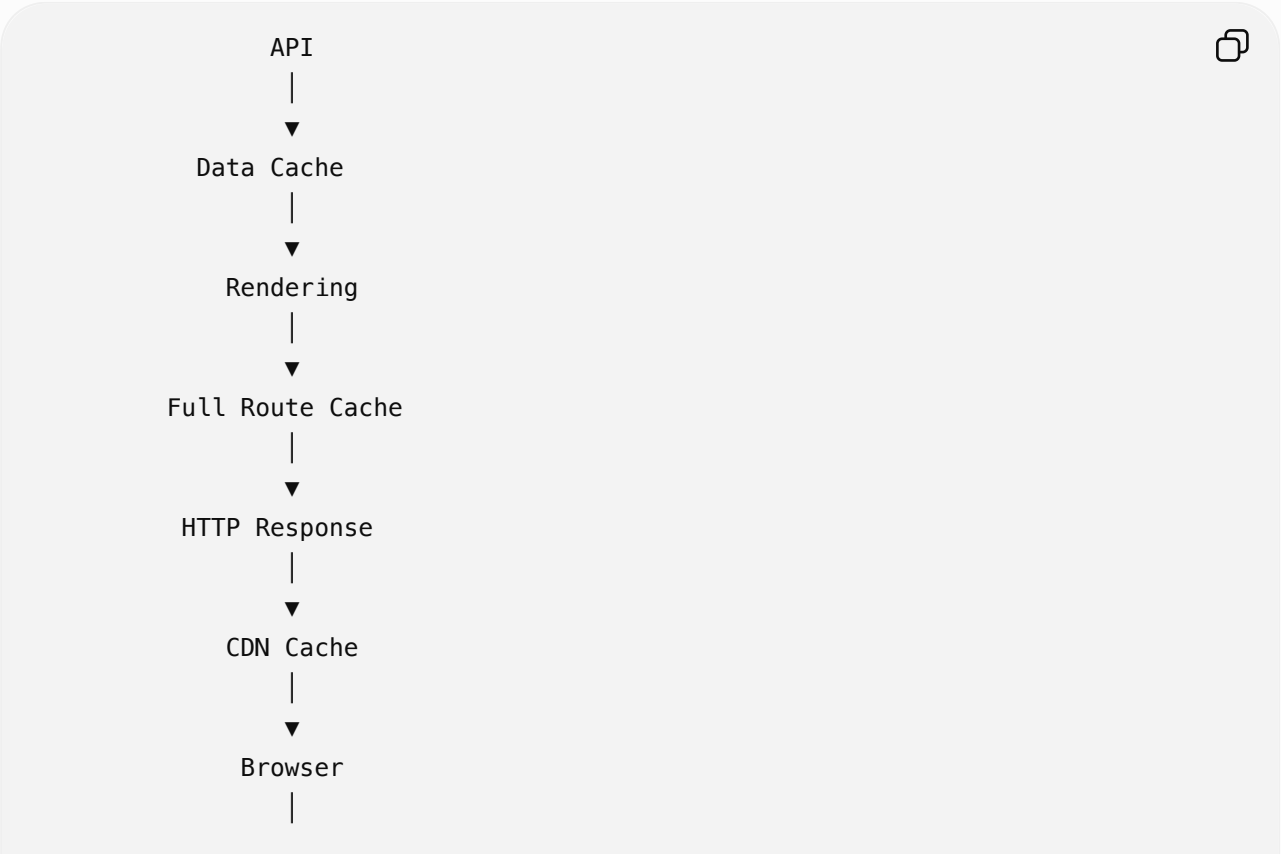
Different situation.

10. Now our three cache layers become clearer

Let's put them side-by-side.

Layer	What it caches	Where
Data Cache	Fetches data	Next.js/server infrastructure
Full Route Cache	Rendered route result	Next.js/server infrastructure
CDN/HTTP cache	HTTP responses	CDN/edge/intermediary
Router Cache	RSC payload	Browser/client

So:



Router Cache

And importantly, **these aren't duplicate copies of exactly the same thing for the same purpose.**

11. One killer example 🔥

Imagine your page:

```
</> TypeScript

export default async function Page() {
  const cookieStore = await cookies()
  const theme = cookieStore.get("theme")?.value

  const response = await fetch(
    "https://api.example.com/products",
    {
      next: { revalidate: 60 }
    }
  )

  const products = await response.json()

  return (
    <div className={theme}>
      ...
    </div>
  )
}
```

Now there are **two different kinds of information:**

Cookie

theme=dark

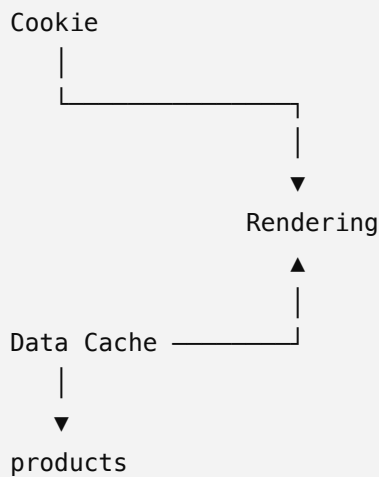
Request-specific.

Products API data

iPhone
MacBook
AirPods

Potentially cacheable/reusable.

So conceptually:



This is a really important insight:

One route can have request-specific information AND separately cacheable data dependencies.

And this becomes **hugely important when we get to Layer 2 / Cache Components**, because modern Next.js gives you more precise control over which parts are cached and which parts remain request-dependent.

Your mental model now

Don't think:

```

Cookie changed
  ↓
Update CDN cache
  
```

Instead:

```

Cookie changed
  ↓
Browser stores new cookie
  ↓
Future request
  ↓
Cookie automatically attached
  ↓
CDN receives request
  ↓
Can cached response safely represent this request?
  |
  | — YES → CDN can serve it
  
```

```
└─ NO → request may need origin/server
      ↓
      Next.js
      ↓
      cookies()
```

That's the whole game. 🎯

And now we're ready for the **Layer 1 debugging exercise**. I'll give you a route and code, and you tell me what happens. I'll deliberately mix **dynamic URL segments**, **cookies**, **searchParams**, **fetch**, **no-store**, **and** **revalidate** so you have to distinguish all of them.



ChatGPT can make mistakes. Check important info.